

Faster network motif discovery by counting isomorphic subtrees*

Joshua A. Grochow[†] and Tarek Tohme[‡]

Abstract. We develop a new algorithm for counting the number of subgraphs of a network isomorphic to a given query graph ($\#SUBGRAPHISOMORPHISM$), motivated by network motif search. High-degree vertices (hubs), common in real-world networks, contribute to a combinatorial explosion in the number of subgraphs, making existing motif search algorithms intractable for motif sizes greater than 7 on a wide variety of networks of interest. Our procedure leverages the k -core decomposition and a novel subtree-counting technique to quickly scan the periphery of a network. These two innovations make our algorithm significantly faster than its predecessors in practice, especially as most real-world networks have a relatively large periphery. We prove that $\#ROOTEDSUBTREEISOMORPHISM$ is $\#P$ -complete via a reduction from counting bipartite matchings. We provide analytic upper bounds on our algorithm’s execution time, and evaluate its performance on 11 real-world networks of varying topologies.

Key words. network motifs, subgraph isomorphism, k -core decomposition

AMS subject classifications. 05C85, 05C30, 05C60

1. Introduction. Networks are useful abstractions for studying structural patterns and properties in many types of systems [20, 32]. One way to study such patterns is by identifying repetitions of certain subnetworks, following the reasoning that in many systems, the frequent occurrence of a “module” is an indicator of its importance in the function of the system. Patterns in a network that appear with statistically significant frequency with respect to a null model (an appropriately defined random network) are called *network motifs* [19]. Examples of networks where motifs have been studied include class diagrams in software architecture [33], brain networks [31], transcriptional regulation networks [30] and many more [1, 12, 21, 23].

Since their introduction [19], an active line of research has been dedicated to the design of algorithms that search for such motifs ever more efficiently. Most of these algorithms fall under one of two categories: network-centric algorithms, in which all [14, 17, 19] or a sample [15, 34] of size- k subgraphs of a given network are enumerated, and motif-centric algorithms [10, 22] in which size- k graphs are enumerated *a priori* and mapped onto a network one by one. Typically in the former category, after enumeration, subgraphs are grouped into isomorphic equivalence classes and their frequencies are then calculated or estimated. Two of the advantages of motif-centric approaches are the possibility of searching for the instances of a single subgraph in a network, and the reduced cost of computing isomorphisms for each subgraph incurred in the classification task of network-centric algorithms. Additional ideas such as symmetry-breaking [10] and expansion trees [22] have enabled further speedups, making it possible to search for all subgraphs up to size 8, and individual subgraphs up to size 30 [10]. The g-

*Submitted to the editors on DATE

Funding: During this work, JAG was partially funded by NSF grant DMS-1622390, and TT was funded by J. A. Grochow startup funds.

[†]Departments of Computer Science and Mathematics, University of Colorado, Boulder (jgrochow@colorado.edu).

[‡]Department of Computer Science, University of Colorado, Boulder (tpt00@mail.aub.edu).

trie, introduced in [26], is an efficient data structure that enables the search for any set of subgraphs, taking advantage of the similarity between subgraphs in that set. This minimizes the cost of redundant searches incurred by motif-centric approaches, which execute as many queries as there are subgraphs, disregarding their potential similarity, while eliminating the unnecessary isomorphism checks of network-centric subgraph enumeration. This method used in combination with symmetry-breaking produces a vast improvement compared with pre-existing approaches [27]. Additional techniques have included focusing only on tree motifs [17], parallelizing subgraph enumeration [29] or subgraph mapping [18].

A few remarks serve to lay out our strategy for an improved motif search algorithm. First, we note that existing approaches to motif search all operate by either enumerating or mapping *instances* of subgraphs inside a network. In principle, to discover motifs it would suffice to count all subgraphs of a network, rather than list them. The counting version of the subgraph isomorphism problem, #SUBGRAPHISOMORPHISM, asks how many subgraphs of a graph G are isomorphic to another graph H . Algorithms for #SUBGRAPHISOMORPHISM include [7] and [2]. The latter color-coding technique achieves polynomial time when counting non-induced subgraphs of bounded treewidth, provided $|H| = O(\log(|G|))$. However it was recently proved [6] that boundedness of the vertex cover number of H is the only condition that guarantees tractability for #SUBGRAPHISOMORPHISM. Regardless of asymptotic behavior, counting subgraphs has the potential to be faster than listing them, owing to the use of clever counting techniques and to the memory access and storage overhead when large numbers of instances are involved. The ESCAPE algorithm [24], which manages to count all size-5 subgraphs orders of magnitude more efficiently than predecessors, is a testimony to this.

Another issue with existing algorithms is that none of them account for the difference in runtime incurred by different subgraph topologies. Early on in the investigation of network motifs, it was suggested [13] that certain subgraphs could contribute disproportionately more to the runtime of motif search because of combinatorial explosions in the number of times they appear in a network. A good example is the star graph of size k , of which there are $\binom{n-1}{k-1}$ instances in a star graph of size $n \geq k$.

Lastly, scale-free networks are known to possess large hierarchically structured hubs [4] which can quickly make the motif search problem intractable in the contexts where such networks show up. Putting aside the discussion of how ubiquitous scale-free networks really are [5], biological networks, where investigations of network motifs originated, often also contain prohibitively large hubs [25] whether their degree distributions are scale-free, geometric, or log-normal. As we will check experimentally, the runtime of motif search can even be dominated by a single star motif while searching among hundreds or thousands of motifs, especially when the target network contains large hubs.

Taking into account the above remarks, we make #SUBGRAPHISOMORPHISM our focus in the present paper, and take advantage of both motif and network topology to accelerate subgraph counting. We build on Grochow-Kellis [10] to design a subgraph-counting algorithm that leverages tree and hub structure in a network. Our main contribution consists in separating each of the network and the query into two subsets using the k -core decomposition: the “periphery”, which contains all the low-connectedness nodes, and the more interconnected nodes, which can be viewed as “the core”. A key feature of this decomposition is that it guarantees that the periphery consists entirely of trees. This enables us to run the peripheries of

both network and query through a fast subtree-counting algorithm, saving considerable time compared with standard motif search procedures. In addition, the k -core decomposition offers a powerful early abort for motif search; the *coreness* of a vertex imposes a strong constraint on the possible isomorphisms that can map a query graph to a target network, allowing our algorithm to waste less time trying out mappings that do not yield valid instances. Complementing this result, we show that $\# \text{ROOTEDSUBTREEISOMORPHISM}$, where the network G and query H are rooted unlabeled trees, is $\# \mathbf{P}$ -complete. This suggests that, although our speed-ups in subtree counting are only heuristic, that one should not expect significantly improved worst-case bounds for the subtree counting portion of our algorithm.

Organization. In Section 2 we make the notions of core and periphery rigorous, provide an algorithm for subtree-counting and analyze its time complexity. Section 3 introduces our main algorithm and provides an analytic treatment of its performance. We then test the algorithm on a corpus of 8 real networks, and relate its empirical performance to several key network statistics in Section 4. In Subsection 4.2 we discuss the impact of subtree-counting and queries of high coreness on the computational complexity of motif search. We conclude in Section 5.

2. Counting subtrees in the 1-shell.

2.1. Background on the k -core decomposition. The k -core decomposition [16] is a procedure that reveals the “core-periphery” structure of a given network. The k -core of a graph G is the (unique) maximal subgraph K such that every vertex in K has at least k neighbors in K . The k -shell is the set of vertices in the k -core that are not part of the $k + 1$ -core. A vertex inside the k -shell is said to be of *coreness* k . The k -core decomposition can be carried out in linear time as follows: remove all vertices of degree 1 until there are none left, including vertices whose degree becomes at most one during this removal process. Those vertices constitute the 1-shell. Make additional passes removing all vertices of degree at most k at each pass, labeling each vertex with its coreness k .

For graphs that have a nonempty 2-core, the 1-shell consists entirely of trees, each of whose root belongs to the 2-core. To see why, notice that no tree can contain a nontrivial subgraph where every vertex is of degree 2 or more. Performing a k -core decomposition on H and G reveals these peripheral trees, if they exist. By selecting one root at a time from each of H and G ’s 1-shell trees, we can then run a subtree-counting algorithm as a subroutine to any standard motif-centric algorithm. We build on the Grochow-Kellis symmetry-breaking algorithm [10], but in principle our subtree-counting technique can be used with any standard motif-centric algorithm. In practice, there are subtleties of implementation that have to do with symmetry-breaking and the 1-shell/2-core boundary that need to be addressed, and will be discussed in subsection 3.1. In the following we discuss the computational complexity of subtree-counting and motivate its use in the context of motif search.

2.2. Description of the subtree-counting algorithm. The permanent of an $m \times n$ matrix $A = (a_{ij})$ is defined as

$$(2.1) \quad \text{per}(A) = \sum_{\sigma \in \Sigma(m,n)} \prod_{i=1}^n a_{i,\sigma(i)}$$

where $\Sigma(m, n)$ is the set of all m -permutations of the index set. In a weighed bipartite graph B with weighed $m \times n$ adjacency matrix $W = (w_{ij})$, if we define the weight of a matching s as the product of the weights of all its edges

$$(2.2) \quad w(s) = \prod_{i=1} w_{i, s(i)}$$

then the permanent of W is the sum of the weights of all matchings of B . If we assign the weight of an edge (u, v) to be the number of matchings between T_u and S_v , the permanent of W becomes the total number of instances of T in S .

If the roots of trees $\{T, S\}$ have k and l children respectively, we need to compute $\binom{l}{k}$ permanents of $(k \times k)$ matrices to count all instances of T in S . Hubs that appear in trees can make $\binom{l}{k}$ a very large number, and permanents of $k \times k$ matrices can be expensive to calculate when k is large. Note that these computations happen for every non-leaf vertex of T , as we elaborate below. The following lemma points to a simplification that solves this issue.

Lemma 2.1. *Let $B = (L, R, W)$ be a directed bipartite graph with an $m \times n$ weighed adjacency matrix $W = (w_{ij})$, and edge direction (l, r) , $\forall (l, r) \in L \times R$. If there exists a set $P \subseteq L$ such that $\forall u \in P$, $\deg(u) = |R|$ and $\forall (u, v) \in S \times R$, $w_{uv} = 1$, then*

$$(2.3) \quad \text{per}(W) = \text{per}(W') \times \binom{|R| - |L \setminus P|}{|P|}$$

where $W' = (w'_{ij})$ is the submatrix of W with entries of weight $w_{ij} > 1$. In other words, if B is the bipartite graph representing the number of matchings between two rooted trees T and S , P corresponds to the depth-1 vertices of T , and their contribution to the total number of matchings is the binomial coefficient in the above expression. $\text{per}(W')$ counts the matchings of T 's rooted subtrees of depth greater than 1 in S .

Proof. W has the form

$$(2.4) \quad W = \begin{pmatrix} 1 & 1 \\ 0 & W' \end{pmatrix}$$

where W' is of dimension $m' \times n'$, and 0 and 1 are respectively the all-zeros and all-ones matrices of appropriate dimension.

$$\begin{aligned} \text{per}(W) &= \sum_{\sigma \in \Sigma(m, n)} \prod_{i=0}^{m-1} w_{i, \sigma(i)} \\ &= \sum_{\sigma \in \Sigma(m, n)} \left[\prod_{i=m-m'}^{m-1} w'_{i, \sigma(i)} \left(\sum_{\sigma \in \Sigma(m, n)} \prod_{j=0}^{m-m'-1} w_{j, \sigma(j)} \right) \right] \\ &= \binom{n-m'}{m-m'} \times \text{per}(W') \end{aligned}$$

Noticing that $|R| = n$, $|L| = m$, and $|L \setminus P| = m'$ leads to the result. ■

150 We use the above to propose a subtree-counting algorithm that achieves drastic speedup
 151 when compared with regular motif search algorithms running on trees. We make use of Ryser's
 152 formula [28] as a black box to compute permanents, but more efficient formulas could easily
 153 be inserted in our algorithm when they become available.

Algorithm 2.1 COUNTROOTEDSUBTREES(T, S, t, s, D)

```

treeChildrenT := {i | deg(i) > 1  ∀i ∈ t.children}
treeChildrenS := {i | deg(i) > 1  ∀i ∈ s.children}
starChildrenT := {i | deg(i) = 1  ∀i ∈ t.children}
subtreesPreorderStrings := {D[i]  ∀i ∈ treeChildrenT}
equivalenceClassSizes := {countStringsEqualTo[k]  ∀k ∈ subtreesPreorderStrings}
multiplicity := 1

for n in equivalenceClassSizes do
  multiplicity := multiplicity × n!
end for

if t.children = ∅ then
  return 1
else if treeChildrenT = ∅ then
  n := t.children.size
  k := s.children.size
  return  $\binom{n}{k}$ 
end if

for u in treeChildrenT do
  for v in treeChildrenS do
     $T_u := \text{detachTree}(T, u, t)$  //the “downward-facing” subtree of  $T$  rooted at  $u$ 
     $S_v := \text{detachTree}(S, v, s)$ 
     $k := \text{COUNTROOTEDSUBTREES}(T_u, S_v, u, v, D)$ 
     $W'[u][v] := k$ 
  end for
end for

n := s.children.size − treeChildrenT.size
k := starChildrenT.size
return (permanent( $W'$ ) ÷ multiplicity) ×  $\binom{n}{k}$ 

```

154 **Algorithm 2.1** counts all the rooted instances of an unlabeled rooted tree T inside an
 155 unlabeled rooted tree S , where the roots of both trees are given by t and s respectively. D is
 156 a dictionary that stores at key u the preorder traversal string of the subtree of T rooted at u
 157 for all nodes $u \in T$ (here we abuse the term “subtree” by having it refer to the subgraph of
 158 T that contains u and all its descendants relative to the root of T).

Among T 's and S 's depth-1 vertices, the subset of those that form a star graph along with the root (i.e. those of degree 1) are called `starChildren`. The remaining vertices are the `treeChildren`. By [Lemma 2.1](#), we can compute matchings among the `treeChildren` of T and S separately, and obtain the matchings between `starChildrenT` and the children of s from the binomial factor.

We are interested in counting only one copy of T inside S for each one of T 's automorphism classes, since we are dealing with unlabeled trees where symmetries do not matter. To account for this, the algorithm starts by listing the preorder strings of the subtrees of T rooted at T 's `treeChildren`, and grouping the identical strings among that list into equivalence classes (recall that a preorder traversal uniquely determines an unlabeled tree, up to symmetry). There are $n!$ possible permutations of the elements of an equivalence class of subtrees, whence the correction factor *multiplicity*.

Next, the algorithm checks for the two base cases, where T consists of only one node with no children, and where T has no `treeChildren`. Otherwise, it loops through every pair of subtrees (T_u, S_v) of depth greater than 1 and recursively counts the instances of T_u in S_v , storing the count in a matrix W' at entry $[u][v]$. This is equivalent to constructing a directed bipartite graph and assigning a weight k to edge (u, v) .

Finally, the permanent of W' is computed using Ryser's formula:

$$(2.5) \quad \text{perm}(W') = \sum_{k=0}^{n-1} (-1)^k \Sigma_k$$

where Σ_k is obtained by summing the products of the row-sums of A_k , the matrix obtained by deleting k rows from W' , for all combinations of rows. The worst-case time complexity of using Ryser's formula for the permanent is $O(2^{n-1}n^2)$ [28]. However, to get an idea of how much faster [Algorithm 2.1](#) is than regular motif search over trees in practice, see [section 4](#).

2.3. Complexity of counting subtrees. We refer to standard textbooks such as Arora & Barak [3] for background on $\#\mathbf{P}$ and $\#\mathbf{P}$ -completeness. Here we remark that $\#\mathbf{P}$ -complete problems are at least as hard, and though to be significantly harder, than NP-complete problems, so a $\#\mathbf{P}$ -completeness result for a counting problem indicates strongly that we should not expect a subexponential-time algorithm to exist. In this section, we prove such a result for the following problem.

#ROOTEDSUBTREEISOMORPHISM

Input: Two rooted trees T and S with their respective roots t and s .

Output: The number of isomorphisms ϕ that map all of T to a subset of S , up to the automorphisms of T , and verify $\phi(t) = s$.

We proceed via a reduction from the known $\#\mathbf{P}$ -complete problem of $\#\text{PERFECTMATCHINGS}$. We include the definition of membership in $\#\mathbf{P}$ [3] below for convenience.

Definition 2.2. A function $f : \{0, 1\}^* \mapsto \mathbb{N}$ is in $\#\mathbf{P}$ if there exists a polynomial-time Turing Machine M such that for any $x \in \{0, 1\}^*$, $f(x)$ is the number of polynomial-size certificates y satisfying $M(x, y) = 1$.

That $\# \text{ROOTEDSUBTREEISOMORPHISM}$ is in $\# \mathbf{P}$ is easy to verify: at each level of both T and S , we sort the children of each vertex by degree and construct the unique map ϕ between T and a subtree of S which preserves that ordering, with $\phi(t) = s$. Namely,

$$(2.6) \quad \forall u_1, u_2 \in T, \deg(u_1) < \deg(u_2) \iff \deg(\phi(u_1)) < \deg(\phi(u_2))$$

We then loop through each $u_i \in T$, and add adjacent vertices $\{v_{ij}\}$ to u_i until $\deg(u_i) = \deg(\phi(u_i))$. The index j indicates the position of the adjacent vertex in the sorted list of u_i 's neighbors. This ensures that the resulting tree T' is isomorphic to S . The sequence of v_{ij} 's is a polynomial-size certificate that uniquely determines the instance of S that T maps to, so there are as many such sequences as there are instances of T in S .

We write $T \preceq S$ if there exists a root-preserving isomorphism between tree T and a subtree of S , and $T \preceq! S$ if such an isomorphism is unique up to the symmetries of T , i.e. there is a unique rooted subtree of S that is rooted-isomorphic to T (note that in a tree, all subgraphs are induced).

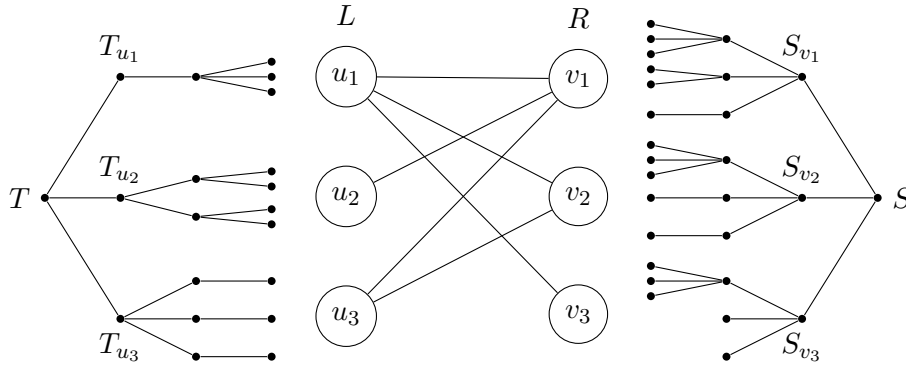


Figure 1. Graphical example of equivalence (2.7).

Theorem 2.3. $\# \text{PERFECTMATCHINGS} \leq_T \# \text{ROOTEDSUBTREEISOMORPHISM}$

Proof. Let $B = (L, R, E)$ be a bipartite graph where L and R are respectively the sets of left and right vertices, and $E \subseteq L \times R$ is the set of edges. We construct a tree T_u for each vertex $u \in L$, and a tree S_v for each $v \in R$ (see Figure 1), such that

$$(2.7) \quad (u, v) \in E \iff T_u \preceq! S_v$$

We construct the $\{T_u\}_{u \in L}$ trees as follows: $\forall i \in \{1, \dots, |L|\}$, T_{u_i} has i depth-1 vertices, and each of those has $|L| - i + 1$ children. For the $\{S_v\}$ trees, $\forall j \in \{1, \dots, |R|\}$, S_{v_j} has exactly $|L|$ depth-1 vertices. We then populate the children of those vertices by “inserting” one copy of T_{u_i} into S_{v_j} if and only if $(u_i, v_j) \in E$; in other words, i among S_{v_j} 's depth-1 vertices has at least $|L| - i + 1$ children, and the remaining $|L| - i$ depth-1 vertices have no children. This construction guarantees that at least one instance of T_{v_i} appears in S_{v_j} whenever $(u_i, v_j) \in E$.

To ensure that at most one instance of T_{v_i} can appear in S_{v_j} for any i and j , we add a labeling for every depth-1 and depth-2 vertex (all leaf vertices and their parents) of both trees

in the spirit of Grochow and Kellis’ symmetry-breaking technique [10]. Namely, for all j , we add a unique label to each of the depth-1 and depth-2 vertices of S_{v_j} . Then, for all i , and for each of levels 1 and 2 in T_{v_i} , we find symmetry-breaking conditions by picking a vertex n_0 at random from that level (which by construction consists of an equivalence class of vertices all isomorphic to each other) and assigning to it the condition $L(n_0) < \min(L(n_1), \dots, L(n_k))$. We do the same for all remaining vertices in that level. Any map from T_{v_i} to S_{v_j} induces a labeling on T_{v_i} , and there is a unique map that satisfies the conditions imposed on these labelings.

To complete the construction, on the left side of the bipartite graph we create a tree by adding an edge between a new root vertex t and the root of T_u , for each $u \in L$. Every T_u is now a subtree of the new tree T , rooted at a child vertex of t . Similarly, we create tree S on the right side. This establishes a bijection between a matching of B and a rooted subtree of S isomorphic to T . The bijection is given as follows:

1. Given a matching $\psi \in (L \times R)^{|L|}$ of B , for each $u \in L$, T_u is isomorphic to exactly one subtree of $S_{\psi(u)}$, and the union of each of these subtrees along with the root s constitutes the unique instance of T that appears within S .
2. Given a rooted subtree $S' \subseteq S$ isomorphic to T , the matching consists of edges $(u_i, v_i) \in L \times R, i \in \{1, \dots, |L|\}$, each corresponding to a map between the subtree $S'_{v_i} \subseteq S'$ and T_{u_i} , the existence of which is guaranteed by the construction.

Valiant has shown that counting the number of perfect matchings is $\#\mathbf{P}$ -complete [?]. Now if a Turing machine can query an oracle for $\#\text{ROOTEDSUBTREEISOMORPHISM}$ on instance (T, S) , the answer is equal to the number of bipartite matchings in graph B , thus establishing the result. ■

Like bipartite matching, this result is surprising considering that its corresponding decision problem, $\text{ROOTEDSUBTREEISOMORPHISM}$, is in $\mathbf{P}$. We note that this result is different from the one proved by Goldberg and Jerrum in 2000 [9], in which counting all rooted subtrees which are *distinct* up to isomorphism within a given tree is proved $\#\mathbf{P}$ -complete, whereas our result concerns counting all rooted subtrees of a tree *equivalent* up to isomorphism to a given tree (disregarding the symmetries of that tree).

3. The subgraph-counting algorithm.

3.1. Boundary conditions. Recall from the introduction that our method consists in separating the 1-shell from the 2-core and performing subtree-counting on the former, then running a regular motif search algorithm on the latter. In doing so we must take care of a few boundary cases that arise at the interface between the 1-shell and the 2-core (**add figure to visualize 1-shell and 2-core**). To see why, let $H = (C_H, P_H)$ be a query graph and $G = (C_G, P_G)$ be a network, where C and P refer respectively to the 2-core and the 1-shell. An instance of H could appear “halfway” between the 1-shell and the 2-core of G such that only part of P_H extends into P_G . Because of this, to properly count every instance of H in G , we must run each tree $T_H \in P_H$ as well as all of its subtrees through [Algorithm 2.1](#). By using the symmetry-breaking technique introduced in [10] on T_H , we can cut down the cost incurred by enumerating all possible subtrees by an amount proportional to the product of the number of symmetries at each level of trees of P_H .

On the flip side, another boundary condition provides us with a powerful early abort when searching for instances of H inside G : suppose that H appears halfway between C_G and P_G , only this time C_H extends partially into P_G . As we prove below, this cannot happen. In fact, it cannot happen between a k -core in H and an h -shell in G for any integers $h < k$ without violating induced isomorphism, which we require in the formal definition of “an instance of H inside G ”.

Definition 3.1. We say that an instance of $H = (V_H, E_H)$ appears in $G = (V_G, E_G)$ when there exists an induced isomorphism $\phi : V_H \mapsto V_G$ between the nodes of H and those of G :

$$(3.1) \quad \forall u, v \in V_H, (u, v) \in E_H \iff (\phi(u), \phi(v)) \in E_G$$

Theorem 3.2. For any k , a mapping between a vertex $v \in H$ of coreness k onto a vertex $u \in G$ of coreness $h < k$ violates induced isomorphism between H and G .

Proof. First notice that a vertex v in a graph’s k -shell can have at most k neighbors in the $k + 1$ -core. Otherwise, it wouldn’t be removed when the k -core decomposition is making passes to remove vertices of degree at most k , and hence would not be part of the k -shell by definition. By the same reasoning, a vertex u in the $k + 1$ -core must have at least $k + 1$ neighbors inside the $k + 1$ -core. The above definition implies that an isomorphism that maps u to v should also map at least one of u ’s $k + 1$ -core neighbors to a neighbor of v in the k -shell. Proceeding by induction, we see that v has at least $k + 1$ neighbors, each of which has at least $k + 1$ neighbors, and so on. Therefore v must be in the $k + 1$ -core, a contradiction. ■

We can use this fact to create an early abort by coreness: our algorithm should never bother to try to map a vertex in H to another in G with lesser coreness.

3.2. Main result. We build on the Grochow-Kellis symmetry-breaking algorithm, and refer to the function from their paper by the same name. **Algorithm 3.1** takes in two graphs H and G , and returns the number of images of H that appear in G . It starts by running a k -core decomposition on both graphs. Then, as in the Grochow-Kellis algorithm, it finds all automorphisms from H onto itself by calling `FINDSUBGRAPHINSTANCES( $H, H$ )` [10], groups the nodes that map to each other in the same equivalence class and returns a representative node from each class. We abbreviate this behavior in the function `findEquivalenceClassRepresentatives`. The symmetry-breaking conditions are also computed similarly as in [10].

The next part of the algorithm consists in listing all the subtrees of the 1-shell of H , up to symmetry. This is done by the function `enumerateSubtrees`, which stores them in the dictionary `subtrees`, keyed by the nodes in H at which they are rooted. It does this by again making use of symmetry-breaking, and listing only one tree from each equivalence class of symmetries.

The algorithm proceeds to count the subtrees of H that appear in G ’s 1-shell and at the boundary with its 2-core. It loops through all of G ’s nodes in the same order as the k -core decomposition to quickly scan the 1-shell nodes and then remove them, making the subsequent 2-core search more tractable. We make use of the function f , which represents the partial isomorphism between a subset of H and G as the algorithm progresses in its search. For the boundary case, we associate each f with a multiplier, which simply consists of a count of the number of subtrees that “extend” from f into the 1-shell of G . `ISOMORPHICEXTENSIONS`

Algorithm 3.1 COUNTSUBGRAPHS(H, G)

```

//Initialization
kCoreDecompose(H)
kCoreDecompose(G)
 $ECR_H := \text{findEquivalenceClassesRepresentatives}(H)$ 
 $SBC_H := \text{findSymmetryBreakingConditions}(H)$ 
 $f := \text{initializeParitalMap}$ 

//enumerating all subtrees in  $H$ 's 1-shell
for  $h$  in  $ECR_H$  do
  if  $h \in P_H$  OR  $h$  is a root then
     $T := \text{detachTree}(H, h[, \text{parent}(h)])$ 
     $\text{subtrees}[h] := \text{enumerateSubtrees}(T, h)$ 
  end if
end for

//counting subtrees in  $G$ 's 1-shell and boundary
for  $g$  in kCoreTraversalOrder( $G$ ) do
  if  $g \in P_G$  AND  $H$  is a tree then
     $S := \text{detachTree}(G, g, \text{parent}(g))$ 
    for  $h$  in  $ECR_H$  do
      1ShellCount := COUNTROOTEDSUBTREES( $H, h, S, g$ )
    end for

  else if  $g$  is a root then
     $S := \text{detachTree}(G, g)$ 
    for  $h$  in  $ECR_H$  do
      for  $T_h$  in  $\text{subtrees}[h]$  do
        boundaryMultiplier := COUNTROOTEDSUBTREES( $T_h, h, S, g$ )
         $f.\text{setMultiplier}(\text{boundaryMultiplier})$ 
         $G.\text{markNodes}(S)$ 
        ISOMORPHICEXTENSIONS( $f, H \setminus T_h, G, SBC_H$ )
      end for
    end for
     $G.\text{removeNodes}(S)$ 
  end if
end for

//counting subgraphs in the 2-core
2CoreCount := count(FINDSUBGRAPHINSTANCES( $H, G$ ))
boundaryCount :=  $\prod_{f \in \text{boundary}} f.\text{getMultiplier}$ 

return 1ShellCount + boundaryCount + 2CoreCount

```

is a function introduced in [10] which incrementally extends f one vertex at a time until it maps the entirety of H onto a subset of G . By using the multiplier, we can initialize f from a root of G and let it expand inward into the 2-core, and use COUNTROOTEDSUBTREES to count partial instances of H in the 1-shell. At the end of the search, we multiply the number of instances found by ISOMORPHICEXTENSIONS by their individual multipliers to recover the total number of boundary instances. The vertices scanned by the subtree-counting routine are marked to prevent f extending into the 1-shell. After all the 1-shell vertices are taken care of, they are removed and only the 2-core of G remains.

Note that here we use a slightly modified version of ISOMORPHICEXTENSIONS that accounts for the early abort by coreness feature discussed above. Namely, when searching for an image vertex $g \in G$, ISOMORPHICEXTENSIONS first checks the coreness of g to ensure that it is greater or equal than the coreness of its tentative preimage h .

3.3. Upper bounds on runtime. The subtree-counting algorithm works by recursively computing the number of matchings at each level of a query tree T and a target tree S . To obtain an analytic upper bound on the number of steps in its execution, we count the total number of calls to COUNTROOTEDSUBTREES, and multiply that by the worst-case runtime of computing the permanent of a weighed adjacency matrix. Note that in general such a matrix will not be square, which leads to an added binomial coefficient in the permanent's runtime contribution. This is because the permanent of a non-square matrix is the product of the permanents of each of its square submatrices.

The following is an approximate expression of the number of steps in COUNTROOTEDSUBTREES' execution on trees T and S . T' is the set of non-leaf vertices of T , ϕ is the root-preserving isomorphism mapping T to a subtree of S , m_u and n_v are the number of children of vertices $u \in T$ and $v \in S$ respectively:

$$(3.2) \quad f_{crs}(T, S) \approx \sum_{u \in T'} \binom{n_{\phi(u)}}{m_u} \times m_u^2 \times 2^{m_u-1}$$

By comparison, the Grochow-Kellis algorithm will test all m vertices of T with all n vertices of S up to symmetries of T .

$$(3.3) \quad f_{gk}(T, S) \approx \frac{1}{|Aut(T)|} \binom{n}{m} = \frac{1}{|Aut(T)|} \binom{1 + \sum_{u \in T'} n_{\phi(u)}}{1 + \sum_{u \in T'} m_u}$$

In the context of counting subgraphs, COUNTROOTEDSUBTREES will be called once for each pair of non-leaf vertices in H and roots in G . To factor in the speedup afforded by the coreness condition described in Theorem 3.2, let $m = |H|$ and $n = |G|$, let m_K, n_K be the sizes of the k -cores of H and G , and R the number of roots in G . This gives an upper bound on the total runtime of the subgraph counting algorithm:

$$(3.4) \quad f_{cs}(H, G) \leq \frac{R}{|Aut(H)|} \times \sum_{T \subseteq P_H} f_{crs}(T, S) \times \binom{n_K}{m_K} \binom{n_{K-1} - m_K}{m_{K-1} - m_K} \cdots \binom{n_2 - m_3}{m_2 - m_3}$$

We compare this with an upper bound on the Grochow-Kellis algorithm’s performance:

$$(3.5) \quad f_{gk}(H, G) \leq \frac{1}{|Aut(H)|} \binom{n}{m}$$

4. Real-data experiments.

4.1. Methods. We implemented the Grochow-Kellis algorithm and Algorithm 3.1 in Python 3.6 and ran them on a set of Intel Xeon E5-2680 v3 processors at 2.50GHz running Red Hat Enterprise Linux 7, from CU Boulder’s RMACC supercomputer. Because each motif/network query is a computation that can be run on a separate processor, we made use of concurrency wherever possible. The runtimes we report in the following section are expressions of the total time taken for all concurrent processes summed together.

Using data from the communityFitNet corpus [8] we tested a sample of 8 undirected networks of comparable sizes (around 150 vertices) spanning different topologies and categories of systems. We took care in our selection (by visual inspection) to make sure each network’s topology was typical of the networks that represent the same type of system in the communityFitNet dataset. In each network we searched for all undirected graphs up to size 8 **see figure**. Informed by the upper bounds derived in the previous section, we then compared the speedup afforded by our algorithm to the following network statistics: size of the 1-shell, maximum degree and average non-leaf degree in the 1-shell, maximum and average coreness of 2-core, and maximum and average tree depth in 1-shell. We identified the first three statistics as being most predictive of a large performance improvement in our algorithm. We computed these statistics for all 442 undirected networks from the communityFitNet corpus, highlighting those with large, dense 1-shells and peripheral hubs as potential applications for our algorithm. Finally, we tested three networks of around 1000 vertices, and demonstrate our algorithm’s performance in querying all undirected graphs up to size 6.

4.2. Results and discussion. Figure 2 shows our selection of 8 undirected networks along with network statistics we considered as potential indicators for our algorithm’s improvement. Figure 3 shows runtimes on three larger networks for additional benchmarking. At first sight, the contribution of subtree-counting to our algorithm’s speedup is more evident than that of early abort by coreness. We found that the speedup gained from queries of high maximal coreness is negligible compared with that of queries of high maximal hub degree (or star graphs). A hint to this lies in the fact that products of binomial coefficients of terms of a sum, such as in (3.4), are vastly larger than sums of binomial coefficients of terms of a sum, as in (3.2), but both are much smaller than binomial coefficients involving the entire sum ((3.3)).

Our aim in these experiments was to provide a common-platform comparison with a standard motif-centric algorithm from the literature. Because our code is written in Python, it was expected that our runtimes are not comparable to those of available motif-finding software, which is often written in faster languages such as Java [10] or C++ [27] and optimized for experimental use. Furthermore, we strongly suspect that the super-exponential runtime on large hubs such as the C. elegans interactome in Figure 2 is due to the combinatorial number of subgraphs being counted, and not to the difficulty in counting them. To illustrate this, we note that the most time-consuming size-8 subgraph to count on the C. elegans interactome

382 network (of which there are 11117) took only 214 seconds for our algorithm. By comparison,
383 the most expensive of all 21 size-5 subgraphs took 2459 seconds for Grochow-Kellis. Thus, a
384 software implementation with less computing overhead per query would greatly contribute to
385 the speedup of our subtree-counting procedure.

386 Our algorithm and subtree-counting technique was designed for undirected graphs, but
387 can easily be extended to include directed and labeled graphs.

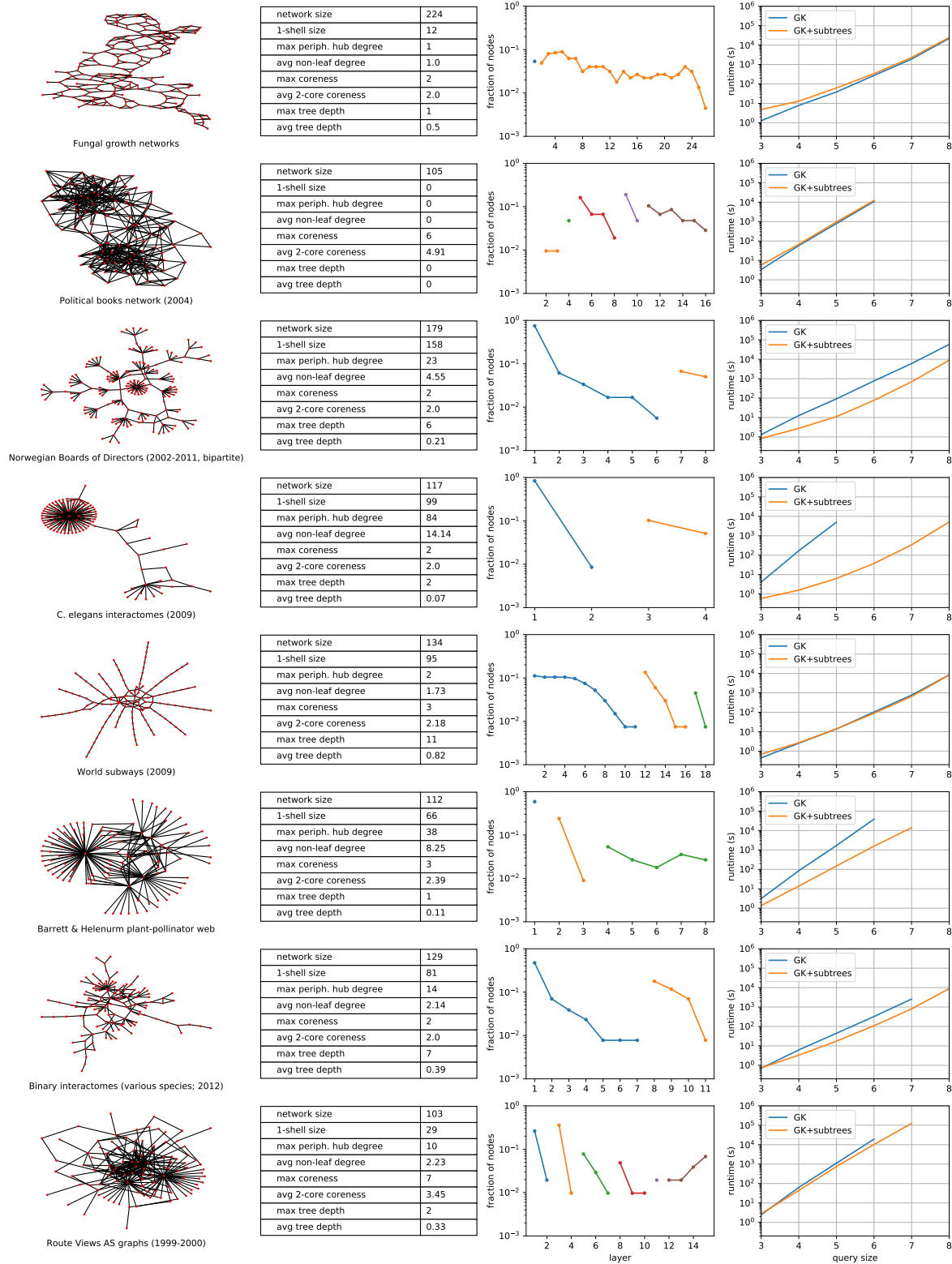


Figure 2. The first column shows each network from our sample. In the second column, max periph. hub degree is the maximum degree of any vertex in the trees of the 1-shell, including their roots. The “degree” of the root is taken as the number of adjacent vertices in the 1-shell. The third column shows the onion spectrum [11] of each network, and the last column shows a runtime comparison between our algorithm and the Grochow-Kellis symmetry-breaking algorithm.

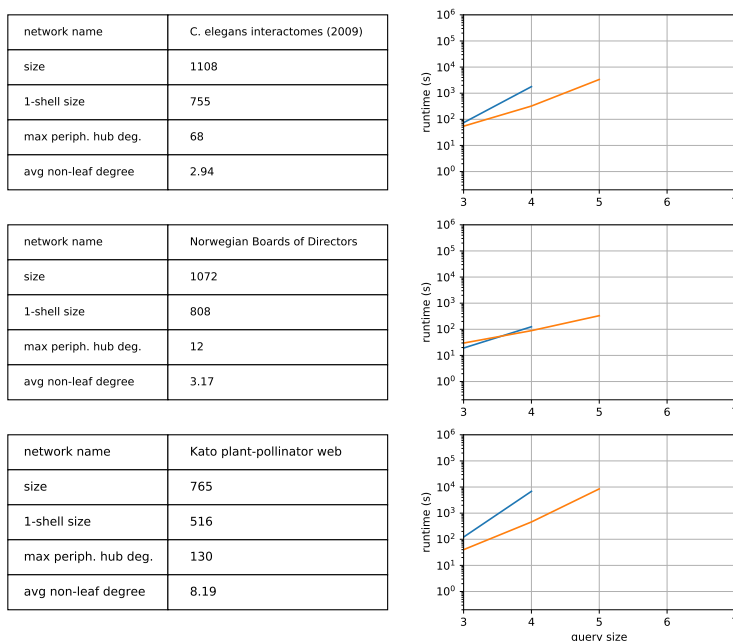


Figure 3.

5. Conclusion. We designed and implemented an algorithm for counting instances of a given graph in a network, leveraging tree and hub structure. Our study suggests that network and query topology can have a great impact on the runtime of motif search algorithms, and alleviates this problem in the case of trees. However, hubs are still major contributors to the complexity of motif search, whether they appear at the peripheries of a network or within the core. A more detailed analysis of the relationship between distribution of runtimes and topologies of query graphs could reveal the best angles of attack for more instance-aware algorithms, which we leave for future work.

REFERENCES

- [1] I. ALBERT AND R. ALBERT, *Conserved network motifs allow protein-protein interaction prediction*, Bioinformatics, 20 (2004), pp. 3346–3352.
- [2] N. ALON, P. DAO, I. HAJIRASOULIHA, F. HORMOZDIARI, AND S. C. SAHINALP, *Biomolecular network motif counting and discovery by color coding*, Bioinformatics, 24 (2008), pp. i241–i249.
- [3] S. ARORA AND B. BARAK, *Computational complexity: a modern approach*, Cambridge University Press, 2009.
- [4] A.-L. BARABÁSI, E. RAVASZ, AND T. VICSEK, *Deterministic scale-free networks*, Physica A: Statistical Mechanics and its Applications, 299 (2001), pp. 559–564.
- [5] A. D. BROIDO AND A. CLAUSET, *Scale-free networks are rare*, Nature communications, 10 (2019), pp. 1–10.
- [6] R. CURTICAPEAN AND D. MARX, *Complexity of counting subgraphs: Only the boundedness of the vertex-cover number counts*, in 2014 IEEE 55th Annual Symposium on Foundations of Computer Science, IEEE, 2014, pp. 130–139.

- [7] F. V. FOMIN, D. LOKSHTANOV, V. RAMAN, S. SAURABH, AND B. R. RAO, *Faster algorithms for finding and counting subgraphs*, Journal of Computer and System Sciences, 78 (2012), pp. 698–706.
- [8] A. GHASEMIAN, H. HOSSEINMARDI, AND A. CLAUSET, *Evaluating and comparing overfit in models of network community structure*, Preprint, (2017).
- [9] L. A. GOLDBERG AND M. JERRUM, *Counting unlabelled subtrees of a tree is $\#P$ -complete*, LMS Journal of Computation and Mathematics, 3 (2000), pp. 117–124.
- [10] J. A. GROCHOW AND M. KELLIS, *Network motif discovery using subgraph enumeration and symmetry-breaking*, in Annual International Conference on Research in Computational Molecular Biology, Springer, 2007, pp. 92–106.
- [11] L. HÉBERT-DUFRESNE, J. A. GROCHOW, AND A. ALLARD, *Multi-scale structure and topological anomaly detection via a new network statistic: The onion decomposition*, Scientific reports, 6 (2016), p. 31708.
- [12] S. ITZKOVITZ, R. LEVITT, N. KASHTAN, R. MILO, M. ITZKOVITZ, AND U. ALON, *Coarse-graining and self-dissimilarity of complex networks*, Physical Review E, 71 (2005), p. 016127.
- [13] S. ITZKOVITZ, R. MILO, N. KASHTAN, G. ZIV, AND U. ALON, *Subgraphs in random networks*, Physical review E, 68 (2003), p. 026127.
- [14] Z. R. M. KASHANI, H. AHRABIAN, E. ELAHI, A. NOWZARI-DALINI, E. S. ANSARI, S. ASADI, S. MOHAMMADI, F. SCHREIBER, AND A. MASOUDI-NEJAD, *Kavosh: a new algorithm for finding network motifs*, BMC bioinformatics, 10 (2009), pp. 1–12.
- [15] N. KASHTAN, S. ITZKOVITZ, R. MILO, AND U. ALON, *Efficient sampling algorithm for estimating subgraph concentrations and detecting network motifs*, Bioinformatics, 20 (2004), pp. 1746–1758.
- [16] Y.-X. KONG, G.-Y. SHI, R.-J. WU, AND Y.-C. ZHANG, *K-core: Theories and applications*, Physics Reports, 832 (2019), pp. 1–32.
- [17] G. LI, J. LUO, Z. XIAO, AND C. LIANG, *Mtmo: an efficient network-centric algorithm for subtree counting and enumeration*, Quantitative Biology, 6 (2018), pp. 142–154.
- [18] W. LIN, X. XIAO, X. XIE, AND X.-L. LI, *Network motif discovery: A gpu approach*, IEEE transactions on knowledge and data engineering, 29 (2016), pp. 513–528.
- [19] R. MILO, S. SHEN-ORR, S. ITZKOVITZ, N. KASHTAN, D. CHKLOVSKII, AND U. ALON, *Network motifs: simple building blocks of complex networks*, Science, 298 (2002), pp. 824–827.
- [20] M. NEWMAN, *Networks*, Oxford university press, 2018.
- [21] T. OHNISHI, H. TAKAYASU, AND M. TAKAYASU, *Network motifs in an inter-firm network*, Journal of Economic Interaction and Coordination, 5 (2010), pp. 171–180.
- [22] S. OMIDI, F. SCHREIBER, AND A. MASOUDI-NEJAD, *Moda: an efficient algorithm for network motif discovery in biological networks*, Genes & genetic systems, 84 (2009), pp. 385–395.
- [23] P. V. PAULAU, C. FEENDERS, AND B. BLASIUS, *Motif analysis in directed ordered networks and applications to food webs*, Scientific reports, 5 (2015), p. 11926.
- [24] A. PINAR, C. SESHADHRI, AND V. VISHAL, *Escape: Efficiently counting all 5-vertex subgraphs*, in Proceedings of the 26th International Conference on World Wide Web, 2017, pp. 1431–1440.
- [25] N. PRŽULJ, D. G. CORNEIL, AND I. JURISICA, *Modeling interactome: scale-free or geometric?*, Bioinformatics, 20 (2004), pp. 3508–3515.
- [26] P. RIBEIRO AND F. SILVA, *G-tries: an efficient data structure for discovering network motifs*, in Proceedings of the 2010 ACM symposium on applied computing, 2010, pp. 1559–1566.
- [27] P. RIBEIRO AND F. SILVA, *G-tries: a data structure for storing and finding subgraphs*, Data Mining and Knowledge Discovery, 28 (2014), pp. 337–377.
- [28] H. J. RYSER, *Combinatorial mathematics*, vol. 14, American Mathematical Soc., 1963.
- [29] Y. SHAO, B. CUI, L. CHEN, L. MA, J. YAO, AND N. XU, *Parallel subgraph listing in a large-scale graph*, in Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, 2014, pp. 625–636.
- [30] S. S. SHEN-ORR, R. MILO, S. MANGAN, AND U. ALON, *Network motifs in the transcriptional regulation network of escherichia coli*, Nature genetics, 31 (2002), pp. 64–68.
- [31] O. SPORNS AND R. KÖTTER, *Motifs in brain networks*, PLoS Biol, 2 (2004), p. e369.
- [32] S. STROGATZ, *Exploring complex networks*, nature, 410 (2001), pp. 268–276.
- [33] S. VALVERDE AND R. V. SOLÉ, *Network motifs in computational graphs: A case study in software architecture*, Physical Review E, 72 (2005), p. 026107.
- [34] S. WERNICKE AND F. RASCHE, *Fanmod: a tool for fast network motif detection*, Bioinformatics, 22

464 (2006), pp. 1152–1153.